LAB 06 CHEATSHEET — Region-Based Segmentation

Bui Quang Chien • 23001837 • MAT3562E Computer Vision 2025–2026

1 K-means — Theory

Objective: Partition N pixels into k clusters minimizing:

$$J = \sum_{j=1}^{k} \sum_{\mathbf{x} \in C_j} \|\mathbf{x} - \boldsymbol{\mu}_j\|^2$$

Algorithm (Lloyd's):

1. Init k centroids (random or K-means++)
2. **Assign:** $\text{label}(\mathbf{x}) = \arg\min_j \|\mathbf{x} - \boldsymbol{\mu}_j\|^2$
3. **Update:** $\boldsymbol{\mu}_j = \frac{1}{|C_j|} \sum_{\mathbf{x} \in C_j} \mathbf{x}$
4. Repeat 2–3 until $\|\Delta \boldsymbol{\mu}\| < \varepsilon$

K-means does NOT guarantee connected regions! Pixels with similar intensity far apart may share a label.

Feature vector variants:
Intensity only: $\mathbf{x} = (I)$ 1-D
Intensity+XY: $\mathbf{x} = (I,\ w \cdot y/H,\ w \cdot x/W)$ 3-D
$w =$ `spatial_weight` controls locality.

2 K-means — Intensity Only

```python
def kmeans_gray_intensity(gray, k=3,
      max_iter=30, tol=1e-4, seed=0):
  X = gray.astype(np.float32).reshape(-1, 1)
  rng = np.random.default_rng(seed)
  idx = rng.choice(X.shape[0], size=k,
                   replace=False)
  centers = X[idx].copy()

  for _ in range(max_iter):
    dists = np.sum(
      (X[:,None,:] - centers[None,:,:]) **2,
      axis=2)
    labels = np.argmin(dists, axis=1)

    new_c = centers.copy()
    for c in range(k):
      pts = X[labels == c]
      if len(pts) > 0:
        new_c[c] = pts.mean(axis=0)

    if np.linalg.norm(new_c - centers) < tol:
      break
    centers = new_c

  return labels.reshape(gray.shape),\
         centers.reshape(-1)
```

Key idea: Flatten image to $N \times 1$, cluster by intensity, reshape labels back.

3 K-means — Intensity + XY

```python
def kmeans_gray_intensity_xy(gray, k=3,
      spatial_weight=0.25, max_iter=30,
      tol=1e-4, seed=0):
  h, w = gray.shape
  yy, xx = np.mgrid[0:h, 0:w]
  I = gray.astype(np.float32) / 255.0
  X = np.stack([
    I,
    spatial_weight * yy.astype(np.float32)
                   / max(h-1, 1),
    spatial_weight * xx.astype(np.float32)
                   / max(w-1, 1)
  ], axis=-1).reshape(-1, 3)
  # ... same loop as intensity-only ...
```

`spatial_weight`	Effect
0.0	Pure intensity (= basic K-means)
0.25	Slight spatial coherence
0.5–0.7	Strong locality, like superpixels
> 1.0	Ignores intensity, grid-like

4 K-means — OpenCV

```python
def kmeans_opencv_intensity(gray, k=2,
                            attempts=10):
  X = gray.reshape(-1,1).astype(np.float32)
  criteria = (cv2.TERM_CRITERIA_EPS +
              cv2.TERM_CRITERIA_MAX_ITER,
              50, 0.2)
  comp, labels, centers = cv2.kmeans(
      X, k, None, criteria, attempts,
      cv2.KMEANS_PP_CENTERS)
  return labels.reshape(gray.shape),\
         centers.reshape(-1), comp
```

Advantages: C++ speed, K-means++ init, multiple `attempts` → more stable. `compactness` = total within-cluster distance (lower is better).

5 Region Growing — Theory

Idea: BFS/DFS from seed pixel; add neighbor if it meets homogeneity criterion.

Two reference strategies:

R1 — Seed Reference:
$|I(x,y) - I_{\text{seed}}| \le T$
Stable but misses gradual gradients.

R2 — Region Mean:
$|I(x,y) - \bar{I}_{\text{region}}| \le T$
Adapts to gradient but may "leak".

Connectivity:

- **4-conn:** up/down/left/right
- **8-conn:** + 4 diagonals (more natural but leaks easier)

```python
from collections import deque

def region_growing_seed_ref(gray, seed,
      threshold=10, connectivity=4):
  h, w = gray.shape
  visited = np.zeros((h,w), dtype=bool)
  region  = np.zeros((h,w), dtype=np.uint8)
  ref_val = float(gray[seed])

  nbrs = [(-1,0),(1,0),(0,-1),(0,1)]
  if connectivity == 8:
    nbrs += [(-1,-1),(-1,1),(1,-1),(1,1)]

  q = deque([seed]); visited[seed] = True
  while q:
    x, y = q.popleft()
    if abs(float(gray[x,y]) - ref_val)\
       <= threshold:
      region[x,y] = 255
      for dx,dy in nbrs:
        nx, ny = x+dx, y+dy
        if 0<=nx<h and 0<=ny<w\
           and not visited[nx,ny]:
          visited[nx,ny] = True
          q.append((nx,ny))
  return region
```

```python
def region_growing_region_mean(gray, seed,
      threshold=10, connectivity=4):
  h, w = gray.shape
  visited = np.zeros((h,w), dtype=bool)
  region  = np.zeros((h,w), dtype=np.uint8)
  region_sum = float(gray[seed])
  region_count = 1
  # ... same BFS but:
  # ref_val = region_sum / region_count
  # when adding pixel:
  #   region_sum += float(gray[nx,ny])
  #   region_count += 1
```

Leak risk: Region mean drifts as the region grows. Use with smaller T near boundaries.

8 Key Parameters

Param	Method	Effect
k	K-means	# of segments
		$\uparrow k$: more detail
`w`	K-means	spatial weight
	+XY	$\uparrow w$: more local
T	Region	tolerance
	Growing	$\uparrow T$: larger region
conn	Region	4 vs 8
	Growing	8: smoother, leakier
seed	Region	start point
	Growing	determines which region

9 Common Mistakes

1. **No preprocessing:** Noise makes K-means and Region Growing unreliable. Always consider Gaussian blur.
2. **Wrong dtype:** OpenCV `kmeans` requires `float32`.
3. **Overflow in distance:** Use `float32` before squaring.
4. **Forgetting visited:** Region Growing without visited array $\rightarrow$ infinite loop.
5. **Seed outside object:** Region Growing result depends entirely on seed position.